Transport Layer Definitions (Chapter 3)

3.1 Introduction & Transport-layer Services

Logical Communication The service provided by the transport layer, giving application processes the illusion that they are directly connected to each other, even though they are on different hosts.
Transport vs. Network Layer • **Transport Layer:** Provides logical communication between *processes*.
• **Network Layer:** Provides logical communication between *hosts*.
Segment The transport-layer packet of data. A segment is created by encapsulating application data with a transport-layer header.
TCP (Transmission Control Protocol) The internet's **reliable, connection-oriented** transport protocol. Provides reliable, in-order byte stream, flow control, and congestion control.
UDP (User Datagram Protocol) The internet's **unreliable, connectionless** transport protocol. Provides a "best-effort" service with minimal overhead.

3.2 Multiplexing and Demultiplexing

Multiplexing At the sender, the process of gathering data from multiple sockets, adding transport-layer headers, and passing the resulting segments to the network layer.
Demultiplexing At the receiver, the process of using header information (port numbers, IP addresses) to deliver an incoming segment to the correct application socket.
Connectionless Demultiplexing (UDP) The receiver uses only the **destination port number** to identify the correct socket. Segments with different source IPs/ports but the same destination port will be directed to the same socket.
Connection-Oriented Demultiplexing (TCP) The receiver uses a **4-tuple** of (source IP, source port, destination IP, destination port) to identify the correct socket. This allows a server to handle multiple, distinct connections from different clients on the same destination port.

3.3 Connectionless Transport: UDP

Best-Effort Service A service model where the protocol makes no guarantees. UDP segments may be lost or delivered out of order.
UDP Checksum An optional field in the UDP header used for error detection. It is calculated using **1's complement arithmetic**.
1's Complement Arithmetic A method of binary addition where any carry-out from the most significant bit is "wrapped around" and added back to the result.

3.4 Principles of Reliable Data Transfer (RDT)

The theoretical foundation for building a reliable protocol on top of an unreliable channel.
Stop-and-Wait The simplest RDT protocol where the sender sends one packet, then stops and waits for a response (ACK/NAK) before sending the next. Highly inefficient.
Error Detection Using a **checksum** to detect if bits in a packet have been corrupted.
Feedback Control messages sent from receiver to sender.
• **ACK (Acknowledgement):** Positive feedback; packet received correctly.
• **NAK (Negative Acknowledgement):** Negative feedback; packet was corrupted.
Sequence Number A number added to a packet to identify it. Used by the receiver to detect duplicate packets, which can occur when an ACK/NAK is lost or garbled.
Duplicate ACK In a NAK-free protocol (like TCP), an ACK for a previously acknowledged packet. It serves as a signal to the sender that the receiver did not get the next expected packet.
Countdown Timer & Timeout A mechanism to handle lost packets. The sender starts a timer after sending a packet. If the timer expires before an ACK is received (a *timeout* event), the sender assumes the packet was lost and retransmits it.
Pipelining The technique of allowing a sender to have multiple, "in-flight," yet-to-be-acknowledged packets in the network at once. Dramatically improves performance over stop-and-wait.
Go-Back-N (GBN) A pipelined protocol where the sender maintains a window of up to N un-ACKed packets. The receiver only sends **cumulative ACKs** for in-order packets. If a packet is lost, the sender retransmits that packet and all subsequent packets in the window.
Selective Repeat (SR) A pipelined protocol where the sender maintains a window of up to N un-ACKed packets. The receiver sends **individual ACKs** for each correctly received packet and buffers out-of-order packets. If a packet is lost, the sender retransmits only that single lost packet.

3.5 Connection-Oriented Transport: TCP

Reliable Byte Stream TCP's service model. Applications see a continuous, ordered stream of bytes, with no message boundaries.
Sequence Number (TCP) The byte-stream number of the *first byte* in a segment's data.
Acknowledgement Number (TCP) The sequence number of the *next byte* the host is expecting to receive. TCP uses **cumulative ACKs**.
Three-Way Handshake The connection establishment process used by TCP to ensure both client and server are ready to communicate.
- **SYN:** The first segment, sent from client to server, to request a connection.
- **SYN-ACK:** The server's response, acknowledging the client's SYN and proposing its own connection parameters.
- **ACK:** The final segment from the client, acknowledging the server's SYN-ACK and completing the connection.

Adaptive Timeout TCP's mechanism for setting its retransmission timeout value by dynamically measuring the RTT.
- **SampleRTT:** The measured time between sending a segment and receiving its ACK.
- **EstimatedRTT:** A smoothed average of SampleRTT values, calculated using an Exponential Weighted Moving Average (EWMA).
- **TimeoutInterval:** The final timeout value, calculated as EstimatedRTT plus a "safety margin" based on the RTT's variability (DevRTT).

Fast Retransmit An optimization that triggers a retransmission after receiving **three duplicate ACKs**, without waiting for a timeout. This allows for much faster recovery from single packet losses.

3.6 & 3.7 Congestion Control

Congestion The state of a network where too many sources are sending too much data too fast for the network (routers) to handle, leading to long delays and packet loss.
Flow Control vs. Congestion Control • **Flow Control:** A 1-to-1 mechanism to prevent a fast sender from overwhelming a slow *receiver*. Managed by the **receive window (rwnd)**.
- **Congestion Control:** A many-to-1 mechanism to prevent senders from overwhelming the *network*. Managed by the **congestion window (cwnd)**.

Congestion Window (cwnd) A sender-side variable that limits the amount of unacknowledged data a TCP sender can have in the network. TCP's sending rate is approximately *cwnd/RTT*.
AIMD (Additive Increase, Multiplicative Decrease) The core TCP congestion control algorithm. It increases 'cwnd' linearly (by 1 MSS per RTT) when the network is not congested, and cuts 'cwnd' in half when a loss event occurs. This creates a "sawtooth" pattern.
Slow Start The initial phase of a TCP connection. 'cwnd' starts at 1 MSS and grows *exponentially* (doubling every RTT) to quickly find the available bandwidth.
Congestion Avoidance The phase entered after Slow Start. 'cwnd' grows *linearly* (by 1 MSS per RTT) to gently probe for more bandwidth.
ssthresh (Slow Start Threshold) The variable that determines at which 'cwnd' value the protocol should switch from Slow Start to Congestion Avoidance. When a loss occurs, 'ssthresh' is typically set to half the current 'cwnd'.
Fast Recovery An optimized state (in TCP Reno) entered after three duplicate ACKs. It allows 'cwnd' to be halved (instead of reset to 1), avoiding a return to Slow Start for minor loss events.

3.8 Evolution of Transport Layer Functionality

TCP CUBIC The default TCP congestion control in Linux. It modifies the window growth function to be a cubic function of time since the last loss event, allowing for much faster recovery on high-speed networks.
Delay-Based Congestion Control A modern approach to congestion control (e.g., TCP BBR) that infers congestion from an *increase in RTT* rather than from packet loss. The goal is to achieve high throughput while maintaining low latency (avoiding bufferbloat).
TCP Fairness The goal that if K TCP sessions share a bottleneck link of bandwidth R, each should receive an average rate of R/K. AIMD naturally converges to a fair state under ideal conditions.